

DAS 839: NoSQL Systems

End-Sem Project Report

April 29, 2025

Submitted by:

Group 15

1. Siddharth Ayathu (IMT2022517)
2. Shreyank G Bhat (IMT2022516)
3. Aayush Bhargav (IMT2022089)
4. Praveen Peter Jay (IMT2022064)

Introduction

This project is a cross-platform data operation handler and logger that supports MySQL, Apache Pig, and MongoDB. It enables users to input and execute database commands specific to each of these systems, seamlessly performing the respective operations and maintaining dedicated operational logs—MysqlLog, PigLog, and MongoLog—for each system. Each operation is tracked with a simulated timestamp using a custom time counter to maintain chronological traceability. Additionally, the system features a powerful merge operation functionality that allows users to specify a source ("from") and a destination ("to") system, enabling the execution and translation of operations from one database environment to another. This unified interface facilitates operation tracking, auditing, and cross-system data manipulation in a streamlined and extensible manner.

1.) MySQL Java Class

The MySQL class is responsible for interacting with a MySQL database and logging the execution details into a file. It provides two main functionalities: executing a query to retrieve data, and executing an update statement to modify data. The results and execution metadata are recorded in a JSON Lines (.jsonl) file named SQLLog.jsonl.

The class maintains two important members: a Connection object named 'conn' that represents an active connection to the database, and a Statement object named 'stmt' used to execute SQL statements. These are initialized through the constructor, which takes a Connection and a Statement as parameters. This setup allows the methods of the class to interact with the database using the provided connection and statement.

Methods

getResult(String query, JsonObject obj, int time) This method executes a SELECT SQL query provided as a string. It iterates through the ResultSet and prints the projected columns of each row to the console. After processing the results, it adds an additional property "time" to the provided JsonObject. The method then appends the JSON object (containing the execution metadata) into the SQLLog.jsonl file. It includes error handling to catch and print SQL and file I/O exceptions without terminating the application.

setResult(String update, JsonObject obj, int time) This method executes an UPDATE SQL command provided as a string. After executing the update, it adds a "time" property to the provided JsonObject and appends it to the SQLLog.jsonl file. Similar to getResult, it handles SQL and general exceptions separately to ensure stability during both database and file operations.

2.) Pig Java Class

The Pig class is responsible for executing Apache Pig scripts locally and managing grade updates stored in a CSV file. It supports running both retrieval and update Pig scripts, while also maintaining a log of operations in a JSON Lines (.jsonl) file called PigLog.jsonl.

The class maintains a constant string baseload command that contains the initial Pig Latin command to load data from grades.csv using PigStorage. This command is automatically prepended to every Pig script executed by the class. There is no explicit constructor; the class uses default construction, and all methods operate relative to the fixed paths for the input and output files.

Methods

getResult(String pigQuery, JsonObject obj, int time) This method is used for executing Pig queries that retrieve data. It writes a temporary Pig script combining the base load command and the user-provided Pig query, executes the script locally using a ProcessBuilder, and prints the output to the console. After execution, it logs the operation along with the execution time into PigLog.jsonl.

setResult(String pigQuery, JsonObject obj, int time) This method is used for executing Pig queries that update data. Similar to getResult, it writes and executes a temporary Pig script. After the script execution, it creates a temporary output csv file and reads the updated grades from the Pig output file located in updated_grades.csv/part-m-00000. It then updates the original grades.csv file by replacing the old grades with the new ones based on "column values". Finally, it deletes the temporary updated_grades.csv directory to clean up. The operation metadata is logged in PigLog.jsonl.

Supporting Methods

runPigScriptForGet(String pigCommands) This private method is responsible for running a Pig script that only retrieves data. It writes the given commands into a temporary .pig file, executes it locally, prints the console output, and deletes the temporary script after execution.

runPigScript(String pigCommands) This private method runs a Pig script intended to update data. In addition to executing the Pig script, it handles reading the updated output, applying changes to grades.csv, and deleting the temporary output files and directories created during the process.

deleteDirectory(File directory) This private method recursively deletes a directory and all its contents. It is used to remove the updated_grades.csv directory after the grade updates have been processed.

3.) Mongo Java Class Explanation

The Mongo class is responsible for interacting with a MongoDB database. It supports both querying and updating operations on MongoDB collections, and logs metadata about each operation into a JSON Lines (.jsonl) file named MongoLog.jsonl.

The class maintains two public members, mongoClient and database, representing the MongoDB client connection and the database to operate on. The constructor initializes these two members using the provided MongoClient and MongoDatabase instances. All operations are performed relative to this database instance.

Methods

getResult(String collectionName, String filterJson, JsonObject obj, String projection, int time) This method is used to query documents from a specified MongoDB collection. It accepts the collection name, a JSON-formatted string representing the filter conditions, and a projection string that defines which fields to include in the result. It parses the filterJson and projection into MongoDB Document objects. Then, it performs a find() operation on the collection with the given filter and projection. The resulting documents are printed to the console in JSON format. After executing the query, it logs the operation details, including the execution time, into the MongoLog.jsonl file. Any exceptions encountered during database operations or file writing are caught and printed.

setResult(String collectionName, String filterJson, String updateJson, JsonObject obj, int time) This method is used to update documents in a specified MongoDB collection. It parses the filterJson to identify which documents to update and parses the updateJson to determine the new values to set. It performs an updateOne operation applying the specified update to the first matching document. A confirmation message is printed to the console showing the filter criteria used. The operation is logged in MongoLog.jsonl with the associated execution time, and any exceptions

are handled appropriately.

Test Case Description

The test cases are designed to evaluate the basic data manipulation operations (GET, SET, and MERGE) across three systems—Pig, MySQL, and MongoDB—on a unified *Grades* table with the schema: **studentID**, **subjectCode**, and **grade**. Each test case consists of a structured JSON command that specifies the operation, database system, and relevant attributes. The operations are summarized as follows:

[illegible]

Figure 1: Test Case Flow Across Pig, MySQL, and MongoDB

- **SET Operation:** The SET command is tested on all three systems to validate the update/insertion of a grade value for a specific student and subject combination. For example, in one Pig SET test, the grade "B" is assigned to a student with a given hashed ID and subject code "EGC 102 / Digital Design". Similar SET operations are issued on MongoDB and MySQL to test consistency of updates across systems.
- **GET Operation:** Multiple GET operations are executed on each database system to retrieve the grade for a given student and subject pair. These operations serve to verify data retrieval accuracy after SET operations. For instance, following a SET on MongoDB, a GET on the same key verifies that the value was correctly written.
- **MERGE Operation:** MERGE commands test the propagation of data between systems. These operations are used to ensure data consistency across databases. For instance, a MERGE from Pig to MySQL implies transferring or synchronizing records from the Pig data store into the MySQL system, and so on for other combinations such as MySQL to MongoDB and MongoDB to Pig. These help verify whether the synchronization logic and transfer scripts maintain the schema and data integrity during migrations.

Each test case ensures that the systems can handle read and write operations reliably and that merge operations maintain data coherence across the three heterogeneous systems.

Oplog Description

The operations logged in the system correspond to different types of database operations carried out across three databases: PIG, MYSQL, and MONGO. Each operation is represented as a JSON object with the following general structure:

[illegible]

Figure 2: OPlog Sample in MongoDB

[illegible]

Figure 3: OPlog Sample in MySQL

```

{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":1}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":2}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":11}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":2}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":11}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":2}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":11}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":14}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["8bcdb96e08acbb8e954f1702af48a21ea05309d90f3fcd3a155a6d86c","IT 989/20 / Thesis / Research hours"],"column_attributes":["grade"],"column_values":["B"],"time":15}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":11}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["0821a9d2c2726e5df4426b6f7a371ce38c2436ad2e566f85f07883c5271c2","TGC 102 / Digital Design"],"column_attributes":["grade"],"column_values":["B"],"time":2}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":11}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":12}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":13}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":13}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":13}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":13}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["f9a274d6da02e480b9b5d801a0037fa39922c4587a245676c96bd434","AIM 511 / Machine Learning"],"column_attributes":["grade"],"column_values":["C"],"time":13}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["8bcdb96e08acbb8e954f1702af48a21ea05309d90f3fcd3a155a6d86c","IT 989/20 / Thesis / Research hours"],"column_attributes":["grade"],"column_values":["B"],"time":14}
{"db":"Pig","op":"SET","table":"Grades","key_attributes":["studentID","subjectCode"],"key_values":["8bcdb96e08acbb8e954f1702af48a21ea05309d90f3fcd3a155a6d86c","IT 989/20 / Thesis / Research hours"],"column_attributes":["grade"],"column_values":["B"],"time":15}

```

Figure 4: OLog Sample in Pig

- **db**: Specifies the database system (e.g., PIG, MYSQL, MONGO) where the operation is being performed.
- **op**: Represents the type of operation being performed. Possible values include:
 - **SET**: Used to update or insert values into a table based on specific key conditions. If the specified key values are found, the corresponding column values are updated or added.
 - **GET**: Used to retrieve specific columns from a table based on a set of key conditions. It allows projecting only the required columns (e.g., grade).
 - **MERGE**: Facilitates the synchronization of data across different database systems. It performs a merge operation where data from one system is integrated into another.
- **table**: Specifies the table in the database on which the operation is being performed (e.g., Grades).
- **key_attributes**: Lists the attributes that form the composite key used to locate specific rows in the table (e.g., studentID, subjectCode).
- **key_values**: Specifies the actual values for the key attributes, which are used to find or update the row.
- **column_attributes**: For SET operations, this field contains the column(s) that will be updated, such as grade.
- **column_values**: For SET operations, this field contains the new values that will be assigned to the specified column attributes.
- **getColumns**: For GET operations, this specifies which columns will be projected or retrieved (e.g., grade).
- **time**: A global timestamp that is used to order operations chronologically, ensuring that the sequence of operations across the system is consistent.

Each operation is logged in a structured format to track the flow of data within and across the systems. The time attribute allows us to maintain the order of operations for accurate analysis, conflict resolution, and debugging.

Test Case Reader

The `testCaseReader` class is responsible for processing and interpreting database operations from a JSONL file. Its functionality can be summarized as follows:

- Takes three dependencies:
 - `SQLClass` for MySQL operations
 - `MongoClass` for MongoDB operations
 - `PigClass` for Apache Pig operations
- Reads the JSONL file line by line, where each line represents a database operation (GET or SET).
- For each operation:
 - Extracts key attributes (e.g., `studentID`, `subjectCode`)
 - Extracts corresponding key values (e.g., '123', 'AIM 511')
 - Builds query conditions dynamically based on key-value pairs
- Handles projection for GET operations:
 - Builds projection strings (e.g., for specific columns like `grade`)
 - Constructs database-specific queries:

- * MySQL: `SELECT` query
- * Pig: `FILTER` statement
- * MongoDB: `find()` with filter and projection
- Executes queries using:
 - * `SQLClass.getResult()`
 - * `PigClass.getResult()`
 - * `MongoClass.getResult()`
- Handles SET operations:
 - Extracts columns and their values to be updated
 - Constructs update queries:
 - * MySQL: `UPDATE` statement
 - * Pig: `FOREACH` statement
 - * MongoDB: `updateOne()` query
 - Executes update operations using:
 - * `SQLClass.setResult()`
 - * `PigClass.setResult()`
 - * `MongoClass.setResult()`
- Adapts dynamically to:
 - Database type (MySQL, Pig, MongoDB)
 - Operation type (GET or SET)
 - Any table and column conditions
- Increments a custom time counter with each operation to simulate a timestamp and maintain operation sequence across system logs.

Merge

Merge Syntax

Each merge operation is specified in the following format:

```
{ "op": "MERGE", "from": "PIG", "to": "MYSQL" }
```

This indicates that all relevant updates present in PIG but missing or outdated in SQL should be applied to MYSQL. The direction is one-way: changes from `from` to `to` only.

Pointer Management

We maintain six pairs of file pointers, one for each directional merge:

Pointer ID	Source to Destination
1	mysql → mongo
2	mysql → pig
3	mongo → mysql
4	mongo → pig
5	pig → mysql
6	pig → mongo

Table 1: Directional file pointers

Initially, all pointers reference the beginning of the respective operation logs. The pair of pointers corresponding to each merge are updated after each merge to the point in the oplog of each system till which operations have been already considered, so as to avoid unnecessary traversal of the entire logs during any subsequent merges of the same type.

Merge Logic

General Procedure

When a merge command is invoked, for example:

```
{ "op": "MERGE", "from": "PIG", "to": "MYSQL" }
```

The pointer pair corresponding to `pig` → `mysql` (pointer ID 5) is used. We place temporary markers at the ends of both PIG and MYSQL oplogs and traverse backwards from those ends until the respective saved pointers are reached, keeping a record of each SET operation performed since the last merge.

Data Comparison Strategy

To determine necessary updates:

- Maintain two hash maps: one for the **from** system (e.g., PIG) and one for the **to** system (e.g., MYSQL).
- Each key in the hashmap is constructed as a concatenation of:
 - `table_name`
 - `key_attributes`
 - `key_values`
 - `column_attributes`
- Values in the hashmap are:
 - `column_values`
 - `time`
- **GET operations are ignored** during this process, since they don't affect the state of the database.

Update Conditions

For each key in the **from** hashmap (e.g., PIG):

- If the key is **not present** in the **to** hashmap (e.g., MYSQL):
 - Perform the corresponding **SET** operation in the **to** system.
 - Append this operation to the **to** oplog.
- If the key **exists in both maps** but with different `column_values`:
 - Compare `time` values.
 - If the **from** system has a later time, perform the **SET** operation in **to** and update its oplog.
- If the `column_values` are identical, do nothing.

Pointer Update

Once merging is complete:

- Update the corresponding pointer pair (e.g., pointer 5) to point to:
 - The end of the updated **to** system's oplog.
 - The end of the **from** system's oplog (which remains unchanged).
- Other pointer pairs remain unchanged until their respective merges are invoked.

Tracking Consistency Across Systems

We model consistency across the three systems using a state diagram, where each node represents a system. An edge from a **from** node to a **to** node indicates that the **to** system has observed all updates made to the **from** system. Since there are three systems, we track the six possible directed relationships between them using six boolean flags, as illustrated in the table below.

Flag Initialization

Initially, all six flags are set to **true**, indicating that all systems are fully consistent with one another immediately after a fresh table load.

Flag Updates on SET Operations

Whenever a **SET** operation occurs in a system, we set the flags corresponding to all outgoing edges from that system to **false**. This reflects the presence of new updates that the other two systems have not yet seen.

Flag Updates on MERGE Operations

Each **MERGE** operation is unidirectional and can also be represented as an edge in the state diagram. When a merge is executed from system A to system B (denoted $A \rightarrow B$), we perform the following steps:

- Set the flag for $A \rightarrow B$ to **true**, as B has now incorporated updates from A.
- Perform two additional logical checks: **Transitivity** and **Correctness**.

Transitivity: Suppose the three systems are A, B, and C, and a merge is performed from A to B. If system C had already seen all updates from A (i.e., the flag for $C \rightarrow A$ is **true**), then by transitivity, we conclude that C has indirectly seen updates now present in B. Therefore, we also set the flag for $C \rightarrow B$ to **true**.

For example, if:

- `mongo_merged_with_pig` is **true**, and
- a new merge sets `pig_merged_with_mysql` to **true**,

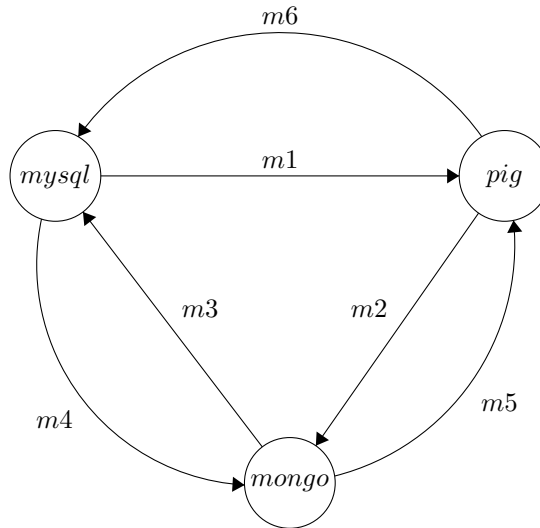
then we can deduce:

$$\text{mongo} \rightarrow \text{pig} \rightarrow \text{mysql}$$

Hence, we also set `mongo_merged_with_mysql` to **true**.

Correctness: When a merge is performed from A to B ($A \rightarrow B$), we also validate downstream consistency. If the flag for $B \rightarrow C$ is **true**, but C has not seen the updates from A now present in B, then B is no longer consistent with C. Consequently, we must set the flag for $B \rightarrow C$ to **false**.

This ensures that consistency is not incorrectly assumed due to outdated indirect merges.



Edge	Boolean flag in code
m1	<code>mysql_merged_with_pig</code>
m2	<code>pig_merged_with_mongo</code>
m3	<code>mongo_merged_with_mysql</code>
m4	<code>mysql_merged_with_mongo</code>
m5	<code>mongo_merged_with_pig</code>
m6	<code>pig_merged_with_mysql</code>

Table 2: Merge variable representations for data sources

Final Synchronization Check

After each merge, if all six flags are `true`, the systems are fully synchronized. In that case:

- Operation logs for all systems are flushed.
- All pointer pairs are reset to the beginning of the respective operation logs.

Formal Properties of Merge

In distributed systems, especially those that adopt eventual consistency models (e.g., CRDTs or multi-master replication), it is critical that the `merge` operation between replicas satisfies specific algebraic properties. These ensure that independently evolving replicas can converge to a consistent state without requiring centralized coordination.

The three foundational properties for such merge operations are:

- **Idempotence:** Repeating a merge operation with the same input does not change the result.
- **Commutativity:** The result of a merge is the same regardless of the order of operands.
- **Associativity:** The grouping of merge operations does not affect the final result.

We analyze how the `merge(from, to)` function in the `MergeHandler` class upholds each of these properties.

Idempotence

$$\text{merge}(X, X) = X$$

Definition: A merge operation is idempotent if applying it multiple times with the same input has no additional effect beyond the first application.

Mechanism: In the `MergeHandler`, this is enforced through a guard clause:

```
if (from.equals(to))  
    return;
```

This ensures that merging a replica with itself is treated as a no-op. The system detects when both operands are the same and exits early without performing any merge actions.

Conclusion: Idempotence is satisfied by construction. It prevents redundant self-merges from modifying state, which is critical under retry mechanisms or at-least-once delivery semantics in distributed environments.

Commutativity

$$\text{merge}(A, B) = \text{merge}(B, A)$$

Definition: The result of a merge is the same regardless of the order of operands. If we do $A \rightarrow B$ or $B \rightarrow A$, even though it is unidirectional, it will result in the same table with respect to the system which calls the merge.

Mechanism: Each replica contains a set of key-value-timestamp tuples:

$$\mathcal{D}_X = \{(k, v, t)\}$$

The merge operation compares entries by key and retains the value with the highest timestamp. This is done independently of merge direction:

- All keys from both sources are unified.
- For each key, the most recent entry (by timestamp) is selected.
- Older or duplicate entries are ignored.

Example: Let:

$$\begin{aligned} A &= \{(x, 1, 10), (y, 2, 15)\} \\ B &= \{(x, 3, 20), (z, 4, 5)\} \end{aligned}$$

Then both:

$$\text{merge}(A, B) = \text{merge}(B, A) = \{(x, 3, 20), (y, 2, 15), (z, 4, 5)\}$$

Conclusion: The merge logic is symmetric with respect to input order, and since only the latest timestamped value is retained per key, the outcome remains identical. Commutativity enables robust synchronization regardless of operation ordering.

Associativity

$$\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$$

Definition: A merge operation is associative if the final result does not depend on how merge operations are grouped.

Mechanism: Each merge combines sets of key-value-timestamp triples and retains the entry with the highest timestamp for each key. The logic consists of:

- Identifying each field uniquely via a computed key.
- Applying last-writer-wins semantics based on the `time` field.
- Overwriting only if the new value is more recent.

This approach ensures that repeated merges always converge to the same state — the one reflecting the latest known updates.

Example: Suppose:

$$A = \{(x, 1, 10)\}$$

$$B = \{(x, 2, 20)\}$$

$$C = \{(x, 3, 30)\}$$

Then:

$$\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C) = \{(x, 3, 30)\}$$

Conclusion: Because timestamp resolution is deterministic and independent of merge grouping, the associative property holds. This supports safe, parallel, and incremental merge sequences in distributed dataflows.

Conclusion

The `merge` operation implemented in `MergeHandler` satisfies the three critical properties for conflict-free replicated data synchronization:

- **Idempotence:** Redundant merges with the same input have no effect.
- **Commutativity:** Order of merging does not influence outcome.
- **Associativity:** Grouping of merge calls does not affect the final state.

Together, these properties ensure deterministic and convergent behavior in distributed systems, enabling eventual consistency across replicas without requiring strict coordination or global ordering.

Contribution

- Siddharth Ayathu - IMT2022517 - 25%
- Shreyank G Bhat - IMT2022516 - 25%
- Aayush Bhargav - IMT2022089 - 25%
- Praveen Peter Jay - IMT2022064 - 25%